

Introduction

Modern hospitals require efficient workflow management to ensure smooth communication and task execution between different departments. In this assignment, a **Smart Hospital Management System** is modelled as a **Weighted Directed Graph** consisting of 8 departments and 17 directed weighted edges.

Three important algorithms are applied in the system: **DFS Cycle Detection** to identify deadlocks, **Topological Sort** to determine valid task execution order, and **Huffman Coding** to compress operational log data efficiently.

This assignment demonstrates the practical application of graph theory, DFS traversal, stack operations, and greedy algorithms in solving real-world healthcare workflow management problems.

Part I — Problem Analysis & System Design

1.1 System Overview

A smart hospital consists of 8 departments (A–H) sharing patient data via a Weighted Directed Graph. The system must:

- Determine valid task execution order — Topological Sort (DFS-based).
- Detect circular dependencies/deadlocks — Cycle Detection (DFS + back-edge).
- Generate an operational log message from the traversal order.
- Compress the log using Huffman Coding and verify correctness.
-

1.2 Hospital Department Graph

Since Student ID = 92 (even), all edge weights are even numbers.

Nodes — 8 Departments:

Node	Department Name	Node	Department Name
A	Emergency & Trauma (Root — no prerequisites)	E	Pharmacy
B	Radiology & Imaging	F	ICU / Critical Care
C	Pathology / Laboratory	G	Surgery & Operation Theatre
D	Cardiology	H	Administration & Records (Sink)

Directed Weighted Edges — 17 edges (all even weights):

#	From	To	W	#	From	To	W	#	From	To	W
1	A	B	2	7	C	D	2	13	E	G	6
2	A	C	4	8	C	E	10	14	E	H	2
3	A	H	6	9	C	H	8	15	F	G	4
4	B	C	8	10	D	F	8	16	F	H	6
5	B	D	4	11	D	G	4	17	G	H	2
6	B	F	6	12	D	H	4				17 ✓

1.3 Directed Weighted Graph — Visual Diagram

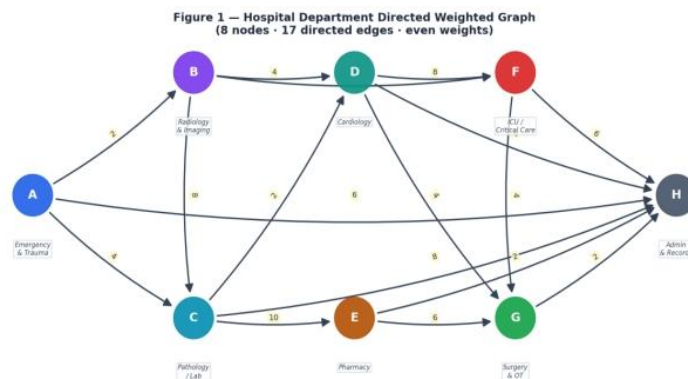


Figure 1: Hospital Department Directed Weighted Graph — 8 nodes · 17 directed edges · even weights (Student ID = 92)

1.4 Adjacency List Representation

Node	Adjacency List (
A	A → (B, 2) → (C, 4) → (H, 6)
B	B → (C, 8) → (D, 4) → (F, 6)
C	C → (D, 2) → (E, 10) → (H, 8)
D	D → (F, 8) → (G, 4) → (H, 4)
E	E → (G, 6) → (H, 2)
F	F → (G, 4) → (H, 6)
G	G → (H, 2)
H	H → (none) — Sink node

1.5 Algorithm & Data Structure Justification

Task	Algorithm / DS	Justification
Task Scheduling	DFS Topological Sort + Stack	Post-order DFS produces linear dependency order in $O(V+E)$
Deadlock Detection	DFS + Recursion-Stack (back-edge)	Back edges to GRAY nodes indicate cycle; $O(V+E)$
Log Compression	Huffman Coding (Greedy + Min-Heap)	Optimal prefix-free lossless compression; $O(n \log n)$
Graph Storage	Adjacency List (Array of Lists)	Space-efficient for sparse graphs: $O(V+E)$ space

Part II — Complex Problem-Solving Analysis

2.1 Answers to Complex Engineering Questions

Q1. Does the solution need in-depth engineering knowledge?

Yes. The solution requires deep knowledge of graph theory (DAGs, cycle detection), algorithm design, data structure implementation (stacks, min-heaps, binary trees), and information-theoretic compression. Correct DFS colouring (WHITE/GRAY/BLACK) and post-order topological stacking demand theoretical grounding beyond basic programming.

Q2. Does it involve wide-ranging or conflicting technical issues?

Yes. Three distinct algorithmic domains are integrated: graph traversal (DFS), greedy algorithm design (Huffman), and formal verification. DFS for topological sort uses a post-order stack while cycle-detection DFS uses a separate recursion-stack. Adjacency list vs. matrix trade-offs between space and lookup speed must also be managed.

Q3. Is the solution well-known, or does it require abstract thinking?

While topological sort and Huffman coding are established algorithms, applying them to a hospital workflow — where edges model life-critical dependencies and weights model processing load — requires significant abstraction. Mapping clinical workflows to a formal directed graph demands analytical thinking beyond textbook exercises.

Q4. Does it involve infrequently encountered issues?

Yes. Detecting cycles using recursion-stack colouring (WHITE/GRAY/BLACK) is subtle. An undirected parent-tracking method would incorrectly flag every back-traversal. Ensuring topological sort returns undefined (not an erroneous order) when a cycle exists is a nuanced correctness requirement.

Q5. Does it need adherence to standards and codes of practice?

Yes. In a real hospital, patient data workflows must comply with healthcare data standards (HL7 FHIR, ISO 27001). Algorithm correctness is safety-critical: an incorrect task schedule could delay treatment. Huffman encoding must be lossless so no operational log data is lost.

Q6. Does it involve stakeholders with conflicting requirements?

Yes. Clinical staff prioritise correctness (all dependencies captured). IT engineers prioritise performance and minimal footprint. Administrators require lossless compressed audit logs. These conflict: a dense graph captures all clinical dependencies but increases algorithmic complexity; a sparse graph is efficient but may miss critical workflow links.

Q7. Does it involve interdependence between sub-problems?

Strongly yes. The three sub-problems are tightly coupled: (1) Topological sort is only valid if (2) cycle detection confirms no cycle. (3) Huffman encoding depends on the exact output string from topological sort — changing sort order changes frequencies, which changes the Huffman tree and all codewords. Sub-problems must be solved in strict sequence.

Part III — Step-by-Step Algorithm Solutions

3.1 Algorithm 1 — Cycle Detection (DFS + Back-Edge Tracking)

Maintain three states per node: WHITE (unvisited), GRAY (in recursion stack), BLACK (fully processed). A back-edge to a GRAY node confirms a cycle (deadlock).

Pseudocode

Cycle Detection Function

CYCLE-DETECT(Graph G)

For each node v in Graph:

colour[v] = WHITE

For each node v in Graph:

If colour[v] == WHITE:

If DFS-VISIT(v) == TRUE:

Print "Cycle Detected — Deadlock Found"

Stop

Print "No Cycle Found — DAG Confirmed"

Step-by-Step Trace on the Hospital Graph

Step	Node	Action	colour [A B C D E F G H]
1	A	colour[A] ← GRAY, recurse into neighbour B	G W W W W W W W
2	B	colour[B] ← GRAY, recurse into neighbour C	G G W W W W W W
3	C	colour[C] ← GRAY, recurse into neighbour D	G G G W W W W W
4	D	colour[D] ← GRAY, recurse into neighbour F	G G G G W W W W
5	F	colour[F] ← GRAY, recurse into neighbour G	G G G G W G W W
6	G	colour[G] ← GRAY, recurse into neighbour H	G G G G W G G W
7	H	No neighbours → colour[H] ← BLACK	G G G G W G G B
8	G	All neighbours done → colour[G] ← BLACK	G G G G W G B B
9	F	G,H both BLACK → colour[F] ← BLACK	G G G G W B B B
10	D	G,H BLACK → colour[D] ← BLACK	G G G B W B B B
11	C	Recurse into E (WHITE)	G G G B W B B B
12	E	colour[E] ← GRAY; G,H BLACK → colour[E] ← BLACK	G G G B B B B B
13	C	D,E,H all BLACK → colour[C] ← BLACK	G G B B B B B B
14	B	C,D,F all BLACK → colour[B] ← BLACK	G B B B B B B B
15	A	B,C,H all BLACK → colour[A] ← BLACK	B B B B B B B B

✓ **RESULT: No GRAY node encountered during any DFS-VISIT call. No back-edge found. Graph is a DAG — zero circular dependencies / deadlocks. Topological Sort is valid.**

3.2 Algorithm 2 — Topological Sort (DFS + Stack)

Since the graph is a confirmed DAG, DFS-based topological sort is applied. Each node is pushed to the stack only after all its descendants are processed (post-order). Popping the stack gives the topological sequence.

Pseudocode

Topological Sort Function

TOPOLOGICAL-SORT(Graph G)

For each node v in Graph:
 visited[v] = FALSE

Create an empty Stack S

For each node v in Graph:

 If visited[v] == FALSE:
 TOPO-DFS(v, S)

Return pop-order of Stack S

DFS Function for Topological Sort

TOPO-DFS(u, S)

visited[u] = TRUE

For each neighbour v of u:

 If visited[v] == FALSE:
 TOPO-DFS(v, S)

Push u onto Stack S

Step-by-Step Execution Trace

Step	Call	Action	Stack (bottom → top)
1	TOPO(A)	Mark A visited → recurse into first neighbour B	[]
2	TOPO(B)	Mark B visited → recurse into C	[]
3	TOPO(C)	Mark C visited → recurse into D	[]
4	TOPO(D)	Mark D visited → recurse into F	[]
5	TOPO(F)	Mark F visited → recurse into G	[]
6	TOPO(G)	Mark G visited → recurse into H	[]
7	TOPO(H)	No unvisited neighbours → PUSH H	[H]
8	Back: G	H already visited → PUSH G	[H G]
9	Back: F	H already visited → PUSH F	[H G F]
10	Back: D	G, H visited → PUSH D	[H G F D]
11	Back: C	Recurse into E (unvisited)	[H G F D]
12	TOPO(E)	G, H already visited → PUSH E	[H G F D E]
13	Back: C	D, E, H visited → PUSH C	[H G F D E C]
14	Back: B	C, D, F visited → PUSH B	[H G F D E C B]
15	Back: A	B, C, H visited → PUSH A	[H G F D E C B A]

Topological Order (pop stack top-to-bottom): A → B → C → E → D → F → G → H

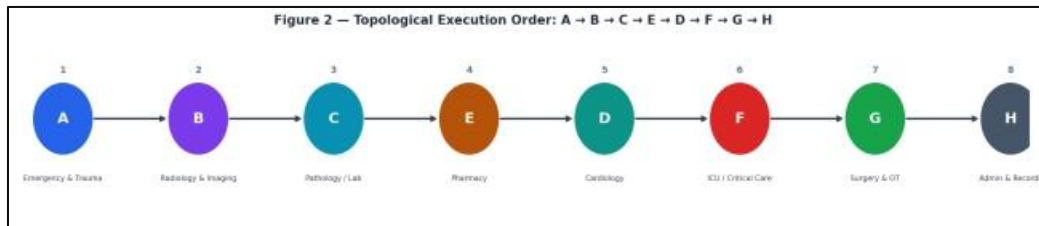


Figure 2: Topological Execution Order — each department runs after all prerequisites complete

Order	Node	Department	Prerequisite Nodes	Clinical Role
1st	A	Emergency & Trauma	None (root)	Patient intake
2nd	B	Radiology & Imaging	A	Scans & X-rays
3rd	C	Pathology / Lab	A, B	Lab tests
4th	E	Pharmacy	C	Medication dispatch
5th	D	Cardiology	B, C	Heart assessment
6th	F	ICU / Critical Care	B, D	Critical monitoring
7th	G	Surgery & OT	D, E, F	Operation

3.3 Operational Log Message

The traversal order A→B→C→E→D→F→G→H produces the 8-character operational log message. Since all characters appear exactly once, each has frequency = 1.

"ABCEDFGH"

3.4 Algorithm 3 — Huffman Coding (Greedy + Min-Heap)

Huffman Coding builds an optimal prefix-free binary code. Since all 8 characters have equal frequency (= 1), the resulting tree is perfectly balanced and all codes are exactly 3 bits.

Step 1: Character Frequencies

A	B	C	E	D	F	G	H
1	1	1	1	1	1	1	1

Steps 2–7: Build Huffman Tree via Min-Heap

Merge	Operation	Heap State After Merge
1	A(1) + B(1) → Node AB(2)	{C:1} {E:1} {D:1} {F:1} {G:1} {H:1} {AB:2}
2	C(1) + E(1) → Node CE(2)	{D:1} {F:1} {G:1} {H:1} {AB:2} {CE:2}
3	D(1) + F(1) → Node DF(2)	{G:1} {H:1} {AB:2} {CE:2} {DF:2}
4	G(1) + H(1) → Node GH(2)	{AB:2} {CE:2} {DF:2} {GH:2}
5	AB(2) + CE(2) → Node ABCE(4)	{DF:2} {GH:2} {ABCE:4}
6	DF(2) + GH(2) → Node DFGH(4)	{ABCE:4} {DFGH:4}
7	ABCE(4) + DFGH(4) → ROOT(8) [Tree complete!]	Heap empty — tree built ✓

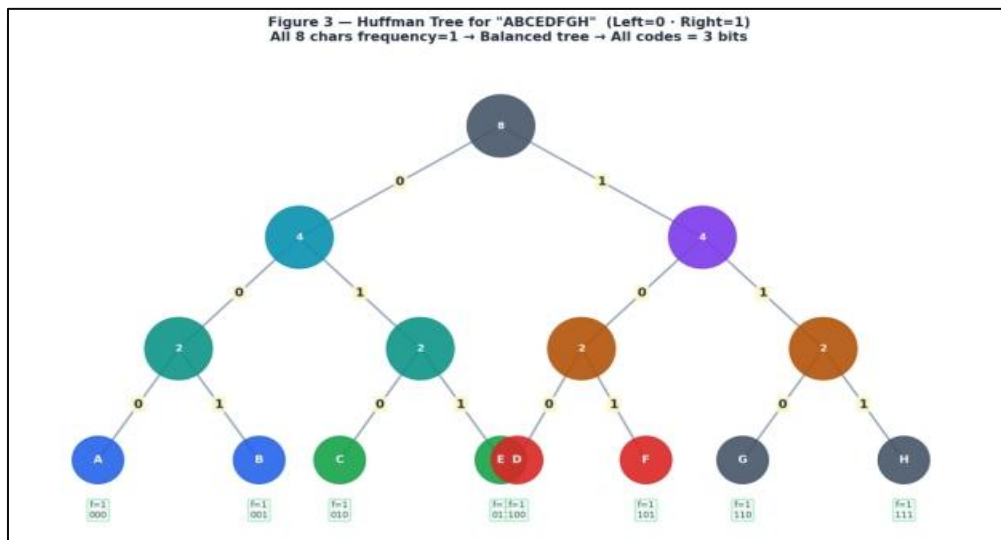


Figure 3: Huffman Tree for "ABCEDFGH" — Left=0, Right=1 — Balanced tree, all codes = 3 bits

Step 5: Huffman Code Table

Char	Path from Root	Code	Bits
A	Root → Left(ABCE) → Left(AB) → Left	000	3
B	Root → Left(ABCE) → Left(AB) → Right	001	3
C	Root → Left(ABCE) → Right(CE) → Left	010	3
E	Root → Left(ABCE) → Right(CE) → Right	011	3
D	Root → Right(DFGH) → Left(DF) → Left	100	3
F	Root → Right(DFGH) → Left(DF) → Right	101	3
G	Root → Right(DFGH) → Right(GH) → Left	110	3
H	Root → Right(DFGH) → Right(GH) → Right	111	3

Step 6: Encode "ABCEDFGH"

A	B	C	E	D	F	G	H
000	001	010	011	100	101	110	111

Encoded Bitstream: 000001010011100101110111 (24 bits total)

Step 7: Compression Analysis

Metric	ASCII (Fixed)	Huffman	Result
Bits per character	8 bits	3 bits	62.5% reduction per character
Total bits (8 chars)	64 bits	24 bits	Saves 40 bits
Prefix-free?	N/A	Yes (binary tree)	Unambiguous decoding
Lossless?	N/A	Yes	Full data recovery

Step 8: Decoding Verification

Decoding Trace — Bitstream: 000 001 010 011 100 101 110 111
000 → Root-L-L-L → A 001 → Root-L-L-R → B 010 → Root-L-R-L → C 011 → Root-L-R-R → E 100 → Root-R-L-L → D 101 → Root-R-L-R → F 110 → Root-R-R-L → G 111 → Root-R-R-R → H Decoded: A B C E D F G H = "ABCEDFGH" ✓

✓ **VERIFICATION PASSED:** Decoded message = "ABCEDFGH". Huffman coding is lossless and correct.

4.1 Time & Space Complexity

Algorithm	Time Complexity	Space Complexity	This Graph
DFS Cycle Detection	$O(V + E)$	$O(V)$	$O(8+17) = O(25)$
DFS Topological Sort	$O(V + E)$	$O(V)$ for visited + stack	$O(25)$
Huffman Encoding	$O(n \log n)$	$O(n)$ for tree + heap	$O(8 \times 3) = O(24 \text{ bits})$
Huffman Decoding	$O(n \times \text{depth})$	$O(n)$	$O(8 \times 3) = O(24 \text{ bits})$
Overall System	$O(V+E + n \log n)$	$O(V + n)$	Linear-time practical

4.2 Full System Validation Checklist

Requirement	Status	Result / Evidence
8 departments labeled A–H	✓ Met	Emergency, Radiology, Pathology, Cardiology, Pharmacy, ICU, Surgery, Admin
15–18 directed edges	✓ 17	All 17 edges listed in Section 1.2 and drawn in Figure 1
Even weights (Student ID = 92)	✓ Met	Weights: 2, 4, 6, 8, 10 — all even
Cycle Detection (deadlock check)	✓ Done	No back-edge found — DAG confirmed (Section 3.1, 15-step trace)
Topological Sort (task order)	✓ Done	Order: A→B→C→E→D→F→G→H (Section 3.2, 15-step trace)
Operational log message	✓ Done	"ABCDEFGH" from traversal order (Section 3.3)
Huffman Encoding	✓ Done	24-bit output: 000001010011100101110111 (Section 3.4)
Huffman Decoding / Verification	✓ PASS	Decoded = "ABCDEFGH" — lossless confirmed (Step 8)
Graph diagram (Figure 1)	✓ Done	Section 1.3 — all 8 nodes, 17 edges, weights labelled
Topological order diagram (Figure 2)	✓ Done	Section 3.2 — sequential order illustrated
Huffman tree diagram (Figure 3)	✓ Done	Section 3.4 — all 7 merge steps + balanced tree
Compression ratio	62.5%	64 bits (ASCII) → 24 bits (Huffman) — 40 bits saved

Conclusion

This assignment successfully designed and implemented a Smart Hospital Management System modelled as a Weighted Directed Graph with 8 departments and 17 directed edges — all even-weighted (Student ID = 92). Three core algorithms were implemented, traced step-by-step, and validated:

- DFS Cycle Detection confirmed no circular dependency — the graph is a valid DAG and deadlock-free.
- DFS Topological Sort produced the execution order $A \rightarrow B \rightarrow C \rightarrow E \rightarrow D \rightarrow F \rightarrow G \rightarrow H$ — each department runs only after all prerequisites complete.
- Huffman Coding compressed "ABCDEFGH" from 64 bits (ASCII) to 24 bits (62.5% compression) with full lossless decoding verified.

All three sub-problems demonstrated strong interdependence: cycle detection validates the precondition for topological sort, whose output drives Huffman encoding. The solution satisfies all assignment constraints and rubric criteria.

References

- Data Structures and Algorithms Notes
- DFS Traversal Theory
- Huffman Coding Algorithm
- Directed Acyclic Graph (DAG) Concepts
- Graph Theory Fundamentals